



Estrutura Completa do Projeto - Cidadão Digital



Visão Geral do Projeto

Status: MVP Phase 1 Completo + 20 Fases Desenhadas + Profiling System Ready

Testado: ☒ End-to-End (React → PHP → MySQL)

Pronto para: Implementação das Fases 2-20 e Deploy Hostinger



Estrutura de Diretórios

```
maiseduc/
├── README_PT_BR.md           # Guia em português para alunos/professor
├── GAME_DESIGN_DOCUMENT.md   # Design completo (20 fases)
├── TIPOS_DE_ALUNO ESTRATEGIAS.md # Pedagogia dos 8 tipos
├── .gitignore                # Segurança: bloqueia .env
├──
├── frontend/                # React + Vite (SPA)
│   ├── package.json          # npm dependencies
│   ├── vite.config.js        # base: './' para Hostinger
│   ├── index.html            # Entry point
│   ├──
│   ├── .env.local            # 🔒 LOCAL (não commitar)
│   │   └── VITE_API_URL=http://localhost:8000/api
│   ├──
│   ├── .env.example          # Template para .env
│   ├──
│   ├── src/
│   │   ├── main.jsx          # React bootstrap
│   │   ├──
│   │   ├── components/
│   │   │   ├── VisualNovelEngine.jsx # Motor do jogo (Fase 1+)
│   │   │   ├── VisualNovelEngine.css # Tema cyberpunk
│   │   │   ├── PerfilAluno.jsx       # 🎨 Exibe perfil (8 tipos)
│   │   │   └── PerfilAluno.css       # Estilo perfil
│   │   ├──
│   │   ├── hooks/
│   │   │   └── useGameProgress.js    # Estado de XP + API calls
│   │   ├──
│   │   └── data/
│   │       ├── fase1.js          # Dados Fase 1 (dialogo + escolhas)
│   │       ├── roadmap20fases.js  # Design de todas as 20 fases
│   │       └── perfil_aluno.js     # Lógica de análise (8 tipos)
│   ├──
│   └── dist/                  # Build output (Hostinger)
```

```

└─ backend/                                # PHP 8.4 API
    └─ .env                                # 🔒 LOCAL (não commitar)
        └─ DB_HOST=localhost
        └─ DB_PORT=3305
        └─ DB_NAME=cidadao_digital_game
        └─ DB_USER=game_app
        └─ DB_PASS=...
    └─ .env.example                        # Template
    └─ .htaccess                           # 🛡 Bloqueia acesso .env
    └─ config/
        └─ database.php                   # PDO connection factory
    └─ api/
        └─ save_decision.php               # POST: Persiste decisão + XP
        └─ calcular_perfil.php             # POST: Calcula tipo de aluno
└─ database/
    └─ schema.sql                         # 📄 DDL completo
        └─ alunos                         # Tabela estudantes
        └─ fases                          # 20 fases seeded
        └─ log_decisoes                   # Cada decisão registrada
        └─ perfil_aluno                   # Tipo + scores
        └─ recomendacoes_professor        # Insights para professor
        └─ Índices para analytics

```

Arquivos Críticos

1. database/schema.sql (215 linhas)

Propósito: Define toda a estrutura do banco

Tabelas principais:

```

alunos (id, nome, id_turma, xp_etica, xp_logica, xp_seguranca, ...)
fases (id, codigo, titulo, competencia_bncc, mecanica, ordem, ...)
log_decisoes (id, id_aluno, id_fase, escolha, classificacao, pontos_ganhos, ...)
perfil_aluno (id, id_aluno, fase_alcançada, tipo_aluno, scores_8_tipos, ...)
recomendacoes_professor (id, id_aluno, tipo_aluno, recomendacao_texto, ...)

```

20 Fases Pre-seeded: Todas as 20 fases estão no `INSERT INTO fases`

Deploy: Copiar completo para `public_html/database/schema.sql` e executar em Hostinger

2. backend/config/database.php (45 linhas)

Propósito: Conexão PDO com fallback seguro

Key Functions:

- `getDbConnection()` : Retorna PDO válido
- `loadEnvFile()` : Lê `.env` local
- Fallback: Se não encontrar env, usa localhost:3305

Deploy: Funciona igual em Hostinger se `.env` correto estiver lá

```
// Uso em qualquer endpoint:
$pdo = getDbConnection();
$stmt = $pdo->prepare('SELECT ... WHERE id = ?');
$stmt->execute([$id]);
```

3. backend/api/save_decision.php (120 linhas)

Propósito: POST endpoint que recebe e persiste decisão

Request Body (JSON):

```
{
  "id_aluno": 1,
  "id_fase": 1,
  "escolha": "verificar_fontes",
  "categoria": "etica",
  "classificacao": "correta",
  "pontos_ganhos": 15
}
```

Operação:

1. Valida payload
2. INSERT INTO log_decisoes
3. UPDATE alunos.xp_[categoria]
4. UPSERT progresso_fases
5. COMMIT ou ROLLBACK

Response: `{ "sucesso": true, "xp_novo": {...} }`

Segurança:

- ☒ Prepared statements
 - ☒ Enum whitelist validation
 - ☒ Generic error messages
 - ☒ Conditional CORS (localhost only)
-

4. backend/api/calcular_perfil.php (150 linhas)

Propósito: Analisa todas decisões e calcula tipo de aluno

Request Body (JSON):

```
{
  "id_aluno": 1,
  "fase_alcancada": 10
}
```

Algoritmo:

1. Fetch todas as `log_decisoos` do aluno
2. Mapear decisões → scores em 8 dimensões (critico, etico, ativista, ...)
3. Normalizar 0-100
4. Encontrar tipo dominante
5. Calcular confiança (diferença entre top 2)
6. Gerar recomendações personalizadas
7. INSERT/UPDATE em `perfil_aluno` e `recomendacoes_professor`

Response:

```
{
  "sucesso": true,
  "tipo_aluno": "investigador",
  "descricao": "Cidadão Investigador 🕵️",
  "confianca": 78,
  "scores": {
    "critico": 45,
    "etico": 60,
    "ativista": 40,
    ...
  },
  "recomendacoes": [
    "Sua análise profunda é excepcional.",
    "Compartilhe suas descobertas: investigação sem comunicação não muda nada."
  ]
}
```

5. frontend/src/components/VisualNovelEngine.jsx (180 linhas)

Propósito: Motor de jogo (renderiza narrativa, processa input)

Props:

```
<VisualNovelEngine
  idAluno={1} // ID do aluno logado
  faseAtual={1} // Qual fase está jogando
```

```
onFaseConcluida={handler} // Callback ao terminar fase  
</>
```

Estado:

```
linhaAtual // Qual linha do diálogo  
textoVisivel // Quanto do texto foi "digitado"  
digitando // Está em animação?  
mostrarEscolhas // Mostrar botões de escolha?  
feedback // Tela de resultado (correta/neutra/catastrofica)
```

Key Functions:

- `avancarDialogo()` : Clique → avança narrativa ou revela texto
- `escolher(decisao)` : Clique em botão → call `useGameProgress.registrarDecisao()`
- Typewriter effect: 22ms por caractere (customizável em CSS)

Output:

- Cena com grid overlay (cyberpunk tema)
- HUD com XP em tempo real
- Caixa de diálogo com nome do personagem
- 3 botões de escolha (com efeito hover)
- Tela de feedback colorida (verde/amarelo/vermelho)

6. frontend/src/hooks/useGameProgress.js (50 linhas)

Propósito: Custom hook que gerencia XP + comunicação com API

Exports:

```
const { xp, enviando, erro } = useGameProgress();  
// xp = { etica: 20, logica: 0, seguranca: 15 }  
// enviando = boolean (true enquanto POST está em flight)  
// erro = null ou string de erro
```

Key Function:

```
await registrarDecisao({  
  idFase: 1,  
  escolha: 'verificar_fontes'  
  // ... (resto das props)  
})  
// Chama POST /api/save_decision.php  
// Atualiza estado XP localmente (optimistic update)
```

Deploy: Lê `VITE_API_URL` do `.env.local` (dev) ou usa `/api` (prod Hostinger)

7. frontend/src/data/fase1.js (80 linhas)

Propósito: Dados narrativos da Fase 1 (desacoplado da engine)

Estrutura:

```
export default {
  id_fase: 1,
  codigo: 'BOLHA_001',
  titulo: 'Prisioneiro da Bolha',
  dialogo: [
    { personagem: 'SISTEMA', texto: '...' },
    { personagem: 'VOCÊ', texto: '...' },
    { personagem: 'ECO-9', texto: '...' },
    // ...
  ],
  escolhas: [
    {
      id_decisao: 'verifica_fonte',
      texto: 'Verificar a procedência',
      classificacao: 'correta',
      pontos_ganhos: { etica: 15, logica: 10, seguranca: 5 },
      feedback: 'Bom! Você...'
    },
    // ...
  ]
}
```

Pattern: Reutilizar para Fases 2-20 (fase2.js , fase3.js , ...)

8. frontend/src/data/roadmap20fases.js (280 linhas)

Propósito: Design completo de todas as 20 fases

Estrutura:

```
const FASES_20 = {
  'UNIDADE_I': [
    {
      id_fase: 1,
      codigo: 'BOLHA_001',
      titulo: 'Prisioneiro da Bolha',
      competencia: 'Letramento Informacional',
      mecanica: 'dilema',
      descricao: '...',
      pilares: ['etica', 'seguranca'],
      tipo_aluno: ['critico', 'investigador']
    },
    // ...
  ]
}
```

```
// ... UNIDADE_II, III, IV, V
}
```

Uso: Referência durante implementação de Fases 2-20

9. frontend/src/data/perfil_aluno.js (320 linhas)

Propósito: Lógica de análise de perfil (8 tipos) + heurísticas

Key Function:

```
analisarPerfilAluno(decisoese) => {
  // Retorna: { tipo_aluno, confianca, scores, recomendacoes }
}

rastrearEvolucaoPerfil(historico_decisoese) => {
  // Retorna evolucao do perfil ao longo das fases
}
```

Matriz de Influência:

- Cada tipo de decisao (verifica_fonte, questiona_autoridade, etc) contribui a diferentes personas
- Normalizacao 0-100
- Calculo de confianca

Deploy: Usado por `calcular_perfil.php` via PHP (logica espelhada)

10. frontend/src/components/PerfilAluno.jsx + .css (120 + 180 linhas)

Propósito: UI que exibe resultado da análise de perfil

Props:

```
<PerfilAluno
  idAluno={1}
  faseAlcancada={10} // Chama calcular_perfil.php nessas props
/>
```

Output:

- Cabeçalho: Emoji + Nome do Tipo + Confianca %
- Grid 8x1: Barras de score para cada tipo
- Recomendações em bullets
- Insight do sistema
- Nota para professor

Flow Completo (End-to-End)

1. Carregamento

```
Browser → http://localhost:5173 (Vite dev)
  → React renderiza <VisualNovelEngine idAluno={1} />
  → Carrega dados de fase1.js
  → Mostra primeira cena
```

2. Jogar Fase 1

```
Aluno lê diálogo (typewriter effect)
  ↓
Clica para avançar
  ↓
Chega em escolhas (3 botões)
  ↓
Clica em "Verificar Fontes" (correta)
  ↓ escolher() chamado
```

3. Salvar Decisão

```
useGameProgress.registrarDecisao({...})
  ↓
POST /api/save_decision.php
  (http://localhost:8000/api/save_decision.php)
  ↓ (enviando=true)
Backend:
- Valida payload
- INSERT log_decisoas
- UPDATE alunos.xp_etica += 15
- COMMIT
  ↓ (resposta JSON)
Hook atualiza estado XP localmente
  ↓
Tela exibe: "Você ganhou 15 XP ÉTICA!"
HUD atualiza: ÉTICA: 20 (no canto superior)
```

4. Terminar Fase 1

```
Aluno clica em "Próxima" na tela de feedback
  ↓
Game component chama onFaseConcluida(1)
  ↓
Aplicação navega para "Escolher Próxima Fase"
```


5. Calcular Perfil (após Fase 4 Boss)

Aplicação chama:

POST /api/calcular_perfil.php
{ id_aluno: 1, fase_alcancada: 4 }
↓

Backend:

- Fetch 30 decisões de log_decisoos
- Aplica matriz de influência
- Normaliza scores
- Encontra tipo_dominante
- Calcula confiança
- INSERT perfil_aluno
- INSERT recomendacoes_professor
- Retorna JSON com tipo + scores + recomendações

↓

PerfilAluno component renderiza resultado

Tecnologia Stack

Layer	Tecnologia	Versão	Nota
Front-end	React	18.3.1	SPA com Vite
Build Tool	Vite	5.4.1	base: './'
Linguagem	JavaScript/JSX	ES2020+	React Hooks
Styling	CSS3	-	Cyberpunk theme
Backend	PHP	8.4.24	PDO para DB
Database	MySQL	8.0.43	Local: porta 3305
Server Dev	PHP built-in	-	localhost:8000
Segurança	PDO Prepared	-	Sem SQL injection
Deploy	Hostinger	-	public_html/

Segurança

- ✔ **Prepared Statements:** Todas queries usam PDO::prepare()
- ✔ **.env Bloqueado:** .htaccess previne acesso HTTP
- ✔ **Validação de Enum:** Whitelist de valores aceitos
- ✔ **Erro Genérico:** Não expõe detalhes internos

- ✓ **CORS Condicional:** Apenas localhost em dev
 - ✓ **.gitignore:** backend/.env nunca é commitado
-

Banco de Dados (Schema)

```
-- Exemplo real após Fase 1
SELECT * FROM alunos WHERE id = 1;
-- id=1, nome='Maria', id_turma=1, xp_etica=20, xp_logica=0, xp_seguranca=0

SELECT * FROM log_decisoos WHERE id_aluno = 1;
-- id=1, id_aluno=1, id_fase=1, escolha='verifica_fontes',
-- classificacao='correta', pontos_ganhos=15, criado_em='2026-09-02 14:23:45'

SELECT * FROM progresso_fases WHERE id_aluno = 1;
-- id_aluno=1, id_fase=1, concluida=1, xp_final=20, concluido_em='2026-09-02 14:24:10'
```

Próximas Etapas

Curto Prazo (1-2 semanas)

- ✓ Criar `fase2.js` (puzzle)
- ✓ Criar UI para puzzle (reconhecer padrões)
- ✓ Testar with MySQL
- ✓ Replicar para Fases 3-4

Médio Prazo (3-4 semanas)

1. Implementar Unidades II-V (Fases 5-20)
2. Criar Dashboard do Professor (UI)
3. Testar profiling com múltiplos alunos
4. Validação pedagógica com professor

Longo Prazo (5-6 semanas)

1. Deploy staging em Hostinger
 2. Testes de carga e performance
 3. Feedback de turma piloto
 4. Iterações de melhorias
 5. Deploy produção
-

Troubleshooting

Erro	Causa	Solução
Cannot POST <code>/api/save_decision.php</code>	PHP server não está rodando	<code>php -S localhost:8000</code>
Can't connect to MySQL	Porta errada	Verificar <code>.env</code> <code>DB_PORT=3305</code>
Access to fetch blocked by CORS	Headers CORS faltando	Verificar CORS condicional em PHP
404 Not Found no build	<code>base: ''</code> errado em vite.config	Usar <code>base: './'</code>
XP não atualiza	<code>useGameProgress</code> não linkado	Verificar <code>VITE_API_URL</code> <code>.env.local</code>

Recursos Adicionais

- **GAME_DESIGN_DOCUMENT.md:** Design das 20 fases
- **TIPOS_DE_ALUNO ESTRATEGIAS.md:** Pedagogia dos 8 tipos
- **README_PT_BR.md:** Guia para alunos/professor
- **.env.example:** Template de variáveis
- **database/schema.sql:** DDL completo

Versão: 1.0 MVP

Data: 02/09/2026

Status:  Pronto para Fase 2 Implementation + Deployment